

Bit Manipulation

Kanishk Goel

7 May, 2026

1 Binary Numbers

Integers are represented as binary numbers by computers. Both signed and unsigned positive integers are represented with their binary digits and negative signed numbers through their Two's complement.

2 Tricks using bitwise operations

- $1 \ll x$ is a number with only the x^{th} bit set.
- $\sim (1 \ll x)$ is a number with only the x^{th} bit not set.
- $n | (1 \ll x)$ sets the x^{th} bit in the number n .
- $n \& \sim (1 \ll x)$ clears the x^{th} bit in the number n .
- $n \wedge (1 \ll x)$ flips the x^{th} bit in the number n .
- $(n \gg x) \& 1$ checks if the x^{th} bit is set.
- n is divisible by 2^k if $n \& (2^k - 1) = 0$
- a predecessor of a power of 2 number n , is the number $n - 1$. It can be used to check if an arbitrary number y is a power of 2, because bitwise and of the number with its predecessor will be 0. So, n is a power of 2 if $n \& (n - 1) = 0$
- A number n , can have its rightmost set bit cleared by $n \& (n - 1)$.
- $n \& (n + 1)$ clears all trailing ones $0011\ 0111_2 \implies 0011\ 0000_2$.
- $n | (n + 1)$ sets the last cleared bit $0011\ 0101_2 \implies 0011\ 0111_2$.
- $n \& -n$ extracts the last set bit $0011\ 0100_2 \implies 0000\ 0100_2$

3 Brian Kernighan's algorithm

Objective: To count the number of set bits in the number n .

Idea: Repeatedly clear the rightmost set bit until the number becomes 0.

4 Counting set bits of all numbers upto n

Idea: For numbers upto 2^x i.e (1 to $2^x - 1$) there are $x \cdot 2^{x-1}$ number of set bits.

This leads to the following algorithm:

1. Find the highest power of 2 lesser than equal to n . Let it be x .
2. Calculate the number of set bits using the above formula.
3. Count the number of set bits in the MSB from 2^x to n and add it.
4. Subtract 2^x from n and repeat the above process.

5 Bitmasks and Enumerating Submasks

A binary digit is used as a flag in bitmasking to denote the status or existence of a feature or trait.

Given a bitmask m , a submask s is one whose set bits are a subset of the set bits of m . To enumerate through all submasks, the idea is simply to start $s = m$ and then modify s as $(s - 1) \& m$.

Justification: $s - 1$ removes the rightmost set bit, and bitwise and with m , matches the set bits of m and $s - 1$. This algorithm, thereby yields descending order of submasks of m .

Biggest application of this is in bitmask dp. Manier times bitmask dp requires iteration through all bitmasks, and for each mask, iteration through all its submasks. This algorithm runs for $m = 0$ to 2^n , for each m , the above submask algorithm is run.

Runtime: $O(3^n)$ - If mask m has k set bits, then it will have 2^k submasks. A total of $\binom{n}{k}$ such masks with k set bits exist. The final expression becomes:-

$$\sum_{k=0}^n \binom{n}{k} \cdot 2^k$$

which by the binomial theorem equals to 3^n .